

MANUAL DE EXPERIMENTOS
LINGUAGEM PYTHON

PARTE II

Ramon Mayor Martins, Ph.D.
ramon.mayor at ifsc.edu.br
IFSC · Câmpus São José

@rmayormartins · rmayormartins.github.io

Pacotes: cálculo, gráfico e estatística
Versão 2026/2

VII. FERRAMENTAS E MATEMÁTICA

17	Ambientes, pacotes e imports	6
18	math: matemática aplicada	8
19	random: sorteio e simulação	6
20	statistics: estatística descritiva	6

VIII. COMPUTAÇÃO NUMÉRICA E GRÁFICOS

21	NumPy: computação vetorial	10
22	Matplotlib: gráficos	10
23	pandas: tabelas de dados	8
24	Projeto: da medida ao relatório	4

IX. APÊNDICES

E	Cartão de consulta científica	1
---	-------------------------------------	---

[0] SOBRE ESTA PARTE II

A Parte I termina na seção 16 e cobre a linguagem. Aqui começa o ferramental: as bibliotecas que transformam Python em instrumento de cálculo, de gráfico e de análise de dados.

A numeração continua de onde parou, da **seção 17 à 24**, mais o apêndice E; a **Parte III** segue da seção 25 à 32, com aprendizado de máquina. Quem já leu a Parte I encontra o mesmo formato: código numerado, saída real do interpretador e quatro experimentos propostos ao fim de cada seção.

A novidade deste volume são as **figuras**. Todo gráfico impresso aqui foi produzido executando o código que aparece logo acima dele, com o mesmo arquivo PNG que o programa grava em disco. Se você digitar o exemplo, verá exatamente a mesma imagem.

O que instalar

Os módulos `math`, `random` e `statistics` já vêm com o Python. Para as seções 21 a 24, instale o restante em um ambiente virtual:

```
[>] TERMINAL
python3 -m venv .venv
source .venv/bin/activate
pip install numpy matplotlib pandas
```

No Google Colab nada disso é necessário: os três pacotes já estão disponíveis. As versões usadas na geração deste volume foram NumPy 2.4, Matplotlib 3.10 e pandas 3.0.

[!] ATENÇÃO

Alguns exemplos das seções 23 e 24 repetem o bloco que monta o conjunto de dados. Isso é proposital: cada código deste manual roda sozinho, sem depender do anterior. Em um script real, esse trecho apareceria uma vez só.

=====

|| BLOCO VII

FERRAMENTAS E MATEMÁTICA

Instalar, importar, calcular e resumir dados.

=====

[17] AMBIENTES, PACOTES E IMPORTS

Antes de usar NumPy ou Matplotlib é preciso saber de onde o Python traz o código de fora. Esta seção fecha o assunto começado na seção 10 da Parte I.

Um **módulo** é um arquivo `.py`. Um **pacote** é uma pasta de módulos. Uma **biblioteca** é o conjunto distribuído por alguém, instalado com `pip`. O `import` procura o nome em uma lista de pastas, na ordem: a pasta do script, os caminhos do ambiente virtual e a instalação do sistema.

+-- #CÓDIGO 17.1 Onde o Python procura os módulos -----

```
import sys

print("versão:", sys.version.split()[0])
print("executável:", sys.executable.split("/")[-1])
print("primeiros caminhos de busca:")
for caminho in sys.path[:4]:
    print(" ", caminho or "(pasta atual)")
```

--- SAÍDA ---

```
versão: 3.12.3
executável: python3
primeiros caminhos de busca:
/tmp/tmpg49pk0mo
/usr/lib/python312.zip
/usr/lib/python3.12
/usr/lib/python3.12/lib-dynload
```

+-- #CÓDIGO 17.2 As formas de importar -----

Escolha uma forma e mantenha o padrão dentro do projeto. O prefixo do módulo custa pouco e deixa claro de onde vem cada função.

```
import math                # nome do modulo como prefixo
import statistics as stat  # apelido
from math import sqrt, pi  # apenas o que sera usado
from math import log10 as db # apelido do que foi importado
```

```
print(math.cos(0), stat.mean([1, 2, 3]))
print(sqrt(16), round(pi, 4), db(1000))
```

Evite "from math import *": sobrescreve nomes sem avisar.

--- SAÍDA ---

```
1.0 2
4.0 3.1416 3.0
```

+-- #CÓDIGO 17.3 Um pacote de verdade -----

O arquivo `__init__.py` transforma a pasta em pacote e é executado na primeira importação.

```
import os

os.makedirs("laboratorio", exist_ok=True)

with open("laboratorio/__init__.py", "w", encoding="utf-8") as f:
    f.write("VERSAO = '1.0'\n")

with open("laboratorio/rf.py", "w", encoding="utf-8") as f:
    f.write("import math\n\n"
            "def watts_para_dbm(p):\n"
            "    return 10 * math.log10(p * 1000)\n\n"
            "def dbm_para_watts(d):\n"
            "    return 10 ** (d / 10) / 1000\n")

from laboratorio import rf, VERSAO

print("pacote versão", VERSAO)
print(rf.watts_para_dbm(0.1), "dBm")
print(round(rf.dbm_para_watts(30), 3), "W")

--- SAÍDA ---
pacote versão 1.0
20.0 dBm
1.0 W
```

+-- #CÓDIGO 17.4 Ambiente virtual e instalação -----

Sem ambiente virtual, um trabalho que precisa de `numpy` 1.x quebra outro que precisa de 2.x. Um ambiente por projeto resolve.

```
# 1. criar o ambiente dentro da pasta do projeto
python3 -m venv .venv

# 2. ativar
source .venv/bin/activate      # Linux e macOS
.venv\Scripts\activate        # Windows

# 3. instalar o basico deste manual
pip install numpy matplotlib pandas

# 4. congelar e reproduzir
pip freeze > requisitos.txt
pip install -r requisitos.txt

# 5. sair do ambiente
deactivate
```

+-- #CÓDIGO 17.5 Conferindo o que está instalado -----

Registre estas versões no relatório de laboratório: resultado numérico sem versão de biblioteca é difícil de reproduzir.

```
from importlib.metadata import version, PackageNotFoundError

for pacote in ["numpy", "matplotlib", "pandas", "pyserial"]:
    try:
        print(f"{pacote:<12} {version(pacote)}")
    except PackageNotFoundError:
        print(f"{pacote:<12} não instalado")

--- SAÍDA ---
numpy          2.4.4
matplotlib     3.10.8
pandas         3.0.2
pyserial       não instalado
```

+-- #CÓDIGO 17.6 Executando módulos com python -m -----

A opção -m executa um módulo como programa e garante que ele venha do mesmo Python que você está usando.

```
python3 -m venv .venv          # cria ambiente
python3 -m pip install numpy    # forma segura de chamar o pip
python3 -m http.server 8000     # servidor de arquivos na pasta
python3 -m json.tool dados.json # formata um JSON
python3 -m timeit "sum(range(1000))"
```

+-----

PACOTE	SERVE PARA	IMPORT HABITUAL
numpy	vetores, matrizes, sinais	import numpy as np
matplotlib	gráficos	import matplotlib.pyplot as plt
pandas	tabelas e séries	import pandas as pd
scipy	filtros, otimização, sinais	from scipy import signal
statistics	estatística sem instalar nada	import statistics
math	matemática escalar	import math

+-- Tabela 17.1: o mínimo para trabalho numérico.

```
[i] SEM INSTALAR NADA

No Google Colab, numpy, matplotlib e pandas já vêm prontos. Para instalar algo extra em uma célula, use !pip install pacote.
```

.....

>> EXPERIMENTOS PROPOSTOS

- . Descubra em qual pasta o seu Python guarda os pacotes instalados usando sys.path.
- . Crie o pacote laboratorio em disco, acrescente o módulo conversoes.py e importe as duas funções.
- . Monte um ambiente virtual e gere o requisitos.txt do seu trabalho.
- . Liste as versões de todas as bibliotecas usadas no seu último relatório.

[18] MATH: MATEMÁTICA APLICADA

O módulo `math` trabalha com números escalares e é a base de qualquer cálculo de engenharia feito em Python.

+-- #CÓDIGO 18.1 Constantes e funções essenciais -----

```
import math

print("pi:", math.pi)
print("e:", math.e)
print("raiz de 2:", math.sqrt(2))
print("hipotenusa 3,4:", math.hypot(3, 4))
print("exponencial:", math.exp(1))
print("valor absoluto:", math.fabs(-7.5))
print("fatorial de 6:", math.factorial(6))
print("mdc e mmc:", math.gcd(24, 36), math.lcm(4, 6))
```

--- SAÍDA ---

```
pi: 3.141592653589793
e: 2.718281828459045
raiz de 2: 1.4142135623730951
hipotenusa 3,4: 5.0
exponencial: 2.718281828459045
valor absoluto: 7.5
fatorial de 6: 720
mdc e mmc: 12 12
```

+-- #CÓDIGO 18.2 Piso, teto e arredondamento -----

`trunc` corta em direção ao zero, `floor` sempre para baixo. E `round` desempata para o par mais próximo.

```
import math

valores = [3.2, 3.7, -3.2, -3.7]

print(f"{'valor':>6}{ 'floor':>7}{ 'ceil':>6}{ 'trunc':>7}{ 'round':>7}")
for v in valores:
    print(f"{'v':>6}{math.floor(v):>7}{math.ceil(v):>6}"
          f"{'math.trunc(v):>7'}{'round(v):>7}'")

print("\nround usa arredondamento bancário:")
print(round(0.5), round(1.5), round(2.5), round(3.5))
```

--- SAÍDA ---

```
valor floor ceil trunc round
  3.2     3     4     3     3
  3.7     3     4     3     4
 -3.2    -4    -3    -3    -3
 -3.7    -4    -3    -3    -4

round usa arredondamento bancário:
0 2 2 4
```

+-+ #CÓDIGO 18.3 Trigonometria e ângulos -----

As funções trigonométricas trabalham em radianos. `atan2(y, x)` devolve o ângulo correto nos quatro quadrantes.

```
import math

for graus in [0, 30, 45, 60, 90]:
    rad = math.radians(graus)
    print(f"{graus}>3}° = {rad:.4f} rad    "
          f"sen={math.sin(rad):.4f} cos={math.cos(rad):.4f}")

print("\narco tangente de (1,1):",
      math.degrees(math.atan2(1, 1)), "graus")
```

```
--- SAÍDA ---

0° = 0.0000 rad    sen=0.0000 cos=1.0000
30° = 0.5236 rad    sen=0.5000 cos=0.8660
45° = 0.7854 rad    sen=0.7071 cos=0.7071
60° = 1.0472 rad    sen=0.8660 cos=0.5000
90° = 1.5708 rad    sen=1.0000 cos=0.0000

arco tangente de (1,1): 45.0 graus
```

+-+ #CÓDIGO 18.4 Logaritmos e decibéis -----

```
import math

def watts_para_dbm(p_watts):
    return 10 * math.log10(p_watts * 1000)

def dbm_para_watts(p_dbm):
    return 10 ** (p_dbm / 10) / 1000

for p in [0.001, 0.01, 0.1, 1.0]:
    print(f"{p}>6} W = {watts_para_dbm(p):>6.1f} dBm")

print("\n30 dBm =", dbm_para_watts(30), "W")
print("ganho de tensão 100x =", 20 * math.log10(100), "dB")
print("log natural, base 2, base 10:",
      math.log(math.e), math.log2(1024), math.log10(1e6))
```

```
--- SAÍDA ---

0.001 W =    0.0 dBm
0.01 W =   10.0 dBm
0.1 W =   20.0 dBm
1.0 W =   30.0 dBm

30 dBm = 1.0 W
ganho de tensão 100x = 40.0 dB
log natural, base 2, base 10: 1.0 10.0 6.0
```

+-- #CÓDIGO 18.5 Precisão: decimal e fractions -----

Use `Decimal` em dinheiro e `Fraction` quando a resposta exata importa mais que a velocidade.

```
from decimal import Decimal, getcontext
from fractions import Fraction

print("float:", 0.1 + 0.2)
print("Decimal:", Decimal("0.1") + Decimal("0.2"))

getcontext().prec = 6
print("divisão com 6 dígitos:", Decimal(1) / Decimal(7))

print("fração:", Fraction(1, 3) + Fraction(1, 6))
print("de float para fração:", Fraction(0.25))
```

```
--- SAÍDA ---
float: 0.30000000000000004
Decimal: 0.3
divisão com 6 dígitos: 0.142857
fração: 1/2
de float para fração: 1/4
```

+-----

+-- #CÓDIGO 18.6 Números complexos e fasores -----

Python trata números complexos como tipo nativo, o que torna cálculo de fasores direto.

```
import cmath
import math

z = complex(3, 4)

print("z =", z)
print("módulo:", abs(z))
print("fase em graus:", math.degrees(cmath.phase(z)))
print("polar:", cmath.polar(z))
print("de volta para retangular:", cmath.rect(5, cmath.phase(z)))

v = cmath.rect(220, math.radians(30)) # tensao 220 V, 30 graus
i = cmath.rect(5, math.radians(-15))  # corrente 5 A
print("impedância:", abs(v / i), "ohms")
```

```
--- SAÍDA ---
z = (3+4j)
módulo: 5.0
fase em graus: 53.13010235415598
polar: (5.0, 0.9272952180016122)
de volta para retangular: (3.0000000000000004+3.9999999999999996j)
impedância: 44.00000000000001 ohms
```

+-----

+-+ #CÓDIGO 18.7 Aplicação: equação do segundo grau -----

```
import math

def raizes(a, b, c):
    """Devolve as raízes reais ou complexas de  $ax^2 + bx + c$ ."""
    delta = b ** 2 - 4 * a * c
    if delta >= 0:
        raiz = math.sqrt(delta)
        return ((-b + raiz) / (2 * a), (-b - raiz) / (2 * a))
    raiz = complex(0, math.sqrt(-delta))
    return ((-b + raiz) / (2 * a), (-b - raiz) / (2 * a))

for coef in [(1, -5, 6), (1, 2, 5), (2, 4, 2)]:
    print(f"{coef} -> {raizes(*coef)}")

--- SAÍDA ---
(1, -5, 6) -> (3.0, 2.0)
(1, 2, 5) -> ((-1+2j), (-1-2j))
(2, 4, 2) -> (-1.0, -1.0)
```

+-+ #CÓDIGO 18.8 Aplicação: equação de Friis -----

Um modelo clássico de enlace escrito com quatro linhas de `math`.

```
import math

def perda_espaco_livre(distancia_km, frequencia_mhz):
    """Perda de propagação em espaço livre, em dB."""
    return (32.44 + 20 * math.log10(distancia_km)
            + 20 * math.log10(frequencia_mhz))

def potencia_recebida(pt_dbm, gt, gr, d_km, f_mhz):
    return pt_dbm + gt + gr - perda_espaco_livre(d_km, f_mhz)

print(f"{d (km)}:>8}{perda (dB)}:>13}{Prx (dBm)}:>12}")
for d in [1, 5, 10, 50]:
    perda = perda_espaco_livre(d, 2400)
    prx = potencia_recebida(20, 12, 2, d, 2400)
    print(f"{d:>8}{perda:>13.2f}{prx:>12.2f}")

--- SAÍDA ---
d (km)  perda (dB)  Prx (dBm)
1       100.04  -66.04
5       114.02  -80.02
10      120.04  -86.04
50      134.02  -100.02
```

>> EXPERIMENTOS PROPOSTOS

- . Escreva funções que convertam entre dB, ganho de potência e ganho de tensão.
- . Calcule a distância entre dois pontos com `math.hypot` e compare com a fórmula manual.
- . Some 0.1 dez vezes com `float` e com `Decimal` e explique a diferença.
- . Calcule a impedância equivalente de um RLC série usando `complex`.

[19] RANDOM: SORTEIO E SIMULAÇÃO

Quando o fenômeno tem incerteza, simular é mais rápido do que deduzir. O módulo `random` gera números pseudoaleatórios com reprodutibilidade controlada.

[!] SORTEIO REPRODUZÍVEL

`random.seed(n)` fixa a sequência gerada. Em trabalho de laboratório, fixe a semente e registre o valor: sem isso ninguém consegue repetir o seu resultado. Para criptografia, use o módulo `secrets`, nunca o `random`.

+-- #CÓDIGO 19.1 Funções básicas -----

```
import random

random.seed(7)

print("float em [0,1):", random.random())
print("inteiro de 1 a 6:", random.randint(1, 6))
print("inteiro em passo de 5:", random.randrange(0, 101, 5))
print("real entre 1.5 e 2.5:", random.uniform(1.5, 2.5))
```

--- SAÍDA ---

```
float em [0,1): 0.32383276483316237
inteiro de 1 a 6: 2
inteiro em passo de 5: 60
real entre 1.5 e 2.5: 2.150934473039854
```

+-- #CÓDIGO 19.2 Amostragem de coleções -----

```
import random

random.seed(7)
equipamentos = ["antena", "cabo", "rádio", "filtro", "fonte"]

print("um item:", random.choice(equipamentos))
print("três sem repetir:", random.sample(equipamentos, 3))
print("três com repetição:", random.choices(equipamentos, k=3))
print("com pesos:", random.choices(["ok", "falha"],
                                   weights=[9, 1], k=10))

baralho = list(range(1, 11))
random.shuffle(baralho)
print("embaralhado:", baralho)
```

--- SAÍDA ---

```
um item: rádio
três sem repetir: ['cabo', 'filtro', 'rádio']
três com repetição: ['antena', 'fonte', 'antena']
com pesos: ['ok', 'falha', 'ok', 'ok', 'ok', 'ok', 'ok', 'ok', 'ok', 'falha']
embaralhado: [5, 3, 4, 6, 8, 2, 9, 7, 1, 10]
```

+-- #CÓDIGO 19.3 Distribuições contínuas -----

gauss gera ruído branco gaussiano, o modelo mais usado para ruído térmico.

```
import random
import statistics

random.seed(7)

normal = [random.gauss(mu=0, sigma=1) for _ in range(10_000)]
expo = [random.expovariate(lambd=0.5) for _ in range(10_000)]

print(f"normal  média {statistics.mean(normal):+.4f} "
      f"desvio {statistics.stdev(normal):.4f}")
print(f"expon.  média {statistics.mean(expo):.4f} "
      f"esperado {1 / 0.5:.4f}")

--- SAÍDA ---
normal  média +0.0045 desvio 0.9977
expon.  média 1.9905 esperado 2.0000
```

+-- #CÓDIGO 19.4 Monte Carlo: estimando pi -----

O erro cai com a raiz do número de amostras: para uma casa decimal a mais, cem vezes mais pontos.

```
import random

def estimar_pi(pontos):
    random.seed(42)
    dentro = 0
    for _ in range(pontos):
        x, y = random.random(), random.random()
        if x * x + y * y <= 1:
            dentro += 1
    return 4 * dentro / pontos

for n in [100, 10_000, 1_000_000]:
    print(f"{n:>9} pontos -> pi ≈ {estimar_pi(n):.5f}")

--- SAÍDA ---
      100 pontos -> pi ≈ 3.12000
    10000 pontos -> pi ≈ 3.12600
  1000000 pontos -> pi ≈ 3.14059
```

+-- #CÓDIGO 19.5 Simulação: taxa de erro de bit -----

```
import random

def simular_enlace(bits, prob_erro):
    random.seed(1)
    erros = sum(1 for _ in range(bits)
               if random.random() < prob_erro)
    return erros, erros / bits

for p in [1e-2, 1e-3, 1e-4]:
    erros, ber = simular_enlace(100_000, p)
    print(f"p={p:<7} erros={erros:<5} BER medida={ber:.5f}")

--- SAÍDA ---
p=0.01   erros=1012  BER medida=0.01012
p=0.001  erros=108   BER medida=0.00108
p=0.0001 erros=5     BER medida=0.00005
```

+-- #CÓDIGO 19.6 Histograma em texto -----

Antes de abrir o Matplotlib, um histograma de terminal já mostra o formato da distribuição.

```
import random

random.seed(3)
lancamentos = [random.randint(1, 6) + random.randint(1, 6)
               for _ in range(1000)]

for face in range(2, 13):
    n = lancamentos.count(face)
    print(f"{face:>3} {'#' * (n // 5):<28} {n:>4}")
```

--- SAÍDA ---

```
2 #####          27
3 #####          55
4 #####          76
5 #####          119
6 #####          156
7 #####          157
8 #####          141
9 #####          121
10 #####          81
11 #####          41
12 #####          26
```

.....
>> EXPERIMENTOS PROPOSTOS

- . Simule 10000 lançamentos de uma moeda viciada com 60% de cara.
- . Estime a probabilidade de duas pessoas fazerem aniversário no mesmo dia em uma turma de 30 alunos.
- . Compare a média de 100, 1000 e 100000 amostras de `random.gauss(0, 1)`.
- . Sorteie a ordem de apresentação dos trabalhos da turma.

[20] STATISTICS: ESTATÍSTICA DESCRITIVA

Média, dispersão, quantis, correlação e regressão sem instalar nada. Para conjuntos de até alguns milhares de pontos, este módulo resolve o relatório inteiro.

+-- #CÓDIGO 20.1 Medidas de posição -----

```
import statistics as stat

medidas = [12.5, 13.0, 11.8, 12.9, 12.1, 13.4, 12.6]

print("média:", round(stat.mean(medidas), 4))
print("mediana:", stat.median(medidas))
print("moda:", stat.mode([1, 2, 2, 3, 3, 3]))
print("média geométrica:", round(stat.geometric_mean([1, 4, 16]), 4))
print("média harmônica:", round(stat.harmonic_mean([40, 60]), 4))
```

--- SAÍDA ---

```
média: 12.6143
mediana: 12.6
moda: 3
média geométrica: 4.0
média harmônica: 48.0
```

+-- #CÓDIGO 20.2 Medidas de dispersão -----

stdev divide por n-1 (amostra) e pstdev divide por n (população). Escolher errado muda o resultado do relatório.

```
import statistics as stat

amostra = [12.5, 13.0, 11.8, 12.9, 12.1, 13.4, 12.6]

print("amplitude:", max(amostra) - min(amostra))
print("variância amostral:", round(stat.variance(amostra), 5))
print("desvio amostral:", round(stat.stdev(amostra), 5))
print("variância populacional:", round(stat.pvariance(amostra), 5))
print("desvio populacional:", round(stat.pstdev(amostra), 5))
```

```
media = stat.mean(amostra)
cv = stat.stdev(amostra) / media * 100
print(f"coeficiente de variação: {cv:.2f}%")
```

--- SAÍDA ---

```
amplitude: 1.5999999999999996
variância amostral: 0.2981
desvio amostral: 0.54598
variância populacional: 0.25551
desvio populacional: 0.50548
coeficiente de variação: 4.33%
```

+-- #CÓDIGO 20.3 Quantis e resumo dos cinco números -----

```
import statistics as stat

dados = [3, 7, 8, 5, 12, 14, 21, 13, 18, 9, 6, 11]
dados.sort()

q1, q2, q3 = stat.quantiles(dados, n=4)
print("mínimo:", min(dados))
print("Q1:", q1)
print("mediana:", q2)
print("Q3:", q3)
print("máximo:", max(dados))
print("intervalo interquartil:", q3 - q1)

limite = q3 + 1.5 * (q3 - q1)
print("possíveis outliers:", [d for d in dados if d > limite])
```

--- SAÍDA ---

```
mínimo: 3
Q1: 6.25
mediana: 10.0
Q3: 13.75
máximo: 21
intervalo interquartil: 7.5
possíveis outliers: []
```

+-- #CÓDIGO 20.4 Correlação e regressão linear -----

Disponível a partir do Python 3.10. Correlação próxima de -1 indica relação inversa forte, como potência recebida contra distância.

```
import statistics as stat

distancia = [1, 2, 3, 4, 5, 6, 7, 8] # km
potencia = [-52, -58, -61, -64, -66, -68, -69, -71] # dBm

print("correlação:", round(stat.correlation(distancia, potencia), 4))
print("covariância:", round(stat.covariance(distancia, potencia), 4))

reta = stat.linear_regression(distancia, potencia)
print(f"modelo: P = {reta.slope:.3f} * d + {reta.intercept:.3f}")

for d in [2.5, 9]:
    print(f"previsto em {d} km:"
          f" {reta.slope * d + reta.intercept:.2f} dBm")
```

--- SAÍDA ---

```
correlação: -0.9696
covariância: -15.0714
modelo: P = -2.512 * d + -52.321
previsto em 2.5 km: -58.60 dBm
previsto em 9 km: -74.93 dBm
```

+-- #CÓDIGO 20.5 Distribuição normal -----

NormalDist entrega densidade, acumulada e percentis sem precisar de tabela impressa.

```
from statistics import NormalDist

ruído = NormalDist(mu=0, sigma=2)    # média 0, desvio 2

print("densidade em 0:", round(ruído.pdf(0), 5))
print("P(X < 2):", round(ruído.cdf(2), 5))
print("P(-2 < X < 2):", round(ruído.cdf(2) - ruído.cdf(-2), 5))
print("percentil 95:", round(ruído.inv_cdf(0.95), 4))

amostra = [11.9, 12.4, 12.1, 12.8, 12.0, 12.5]
ajuste = NormalDist.from_samples(amostra)
print(f"ajuste: mu={ajuste.mean:.4f} sigma={ajuste.stdev:.4f}")

--- SAÍDA ---
densidade em 0: 0.19947
P(X < 2): 0.84134
P(-2 < X < 2): 0.68269
percentil 95: 3.2897
ajuste: mu=12.2833 sigma=0.3430
```

+-- #CÓDIGO 20.6 Relatório estatístico completo -----

```
import statistics as stat

def resumo(nome, dados):
    q1, q2, q3 = stat.quantiles(dados, n=4)
    linhas = [
        ("n", len(dados)),
        ("média", round(stat.mean(dados), 3)),
        ("mediana", round(q2, 3)),
        ("desvio", round(stat.stdev(dados), 3)),
        ("mínimo", min(dados)),
        ("máximo", max(dados)),
        ("Q1 / Q3", f"{q1:.2f} / {q3:.2f}"),
    ]
    print(f"== {nome} ==")
    for rotulo, valor in linhas:
        print(f"{rotulo:<10}{valor}")
    print()

resumo("RSSI antena A", [-52, -58, -61, -54, -66, -59, -57, -60])
resumo("RSSI antena B", [-48, -50, -49, -51, -47, -52, -50, -49])

--- SAÍDA ---
== RSSI antena A ==
n            8
média       -58.375
mediana     -58.5
desvio       4.307
mínimo      -66
máximo      -52
Q1 / Q3     -60.75 / -54.75

== RSSI antena B ==
n            8
média       -49.5
mediana     -49.5
desvio       1.604
mínimo      -52
máximo      -47
Q1 / Q3     -50.75 / -48.25
```

.....

>> EXPERIMENTOS PROPOSTOS

- . Calcule média, mediana e desvio de dez medidas do seu laboratório e diga qual medida melhor representa o conjunto.
- . Identifique outliers em um conjunto usando o critério do intervalo interquartil.
- . Ajuste uma reta a um conjunto de pontos e calcule o erro em cada ponto.
- . Use `NormalDist` para achar a probabilidade de uma medida cair fora de dois desvios padrão.

=====

|| BLOCO VIII

COMPUTAÇÃO NUMÉRICA E GRÁFICOS

Arrays, figuras, tabelas e um relatório completo.

=====

[21] NUMPY: COMPUTAÇÃO VETORIAL

O array do NumPy substitui listas em cálculo numérico. Uma operação escrita sobre o array inteiro roda em código compilado, o que muda a escala do que dá para fazer em sala.

```
[>] IMPORT PADRÃO
import numpy as np. O apelido np é convenção universal: use-o para que qualquer pessoa reconheça o seu código.
```

+-- #CÓDIGO 21.1 Criando arrays -----

arange usa passo, linspace usa quantidade de pontos. Para eixo de tempo, quase sempre linspace.

```
import numpy as np

a = np.array([1, 2, 3, 4])
zeros = np.zeros(5)
uns = np.ones((2, 3))
faixa = np.arange(0, 10, 2)
pontos = np.linspace(0, 1, 5)      # inclui o extremo

print(a)
print(zeros)
print(uns)
print(faixa)
print(pontos)
```

--- SAÍDA ---

```
[1 2 3 4]
[0. 0. 0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[0 2 4 6 8]
[0.    0.25 0.5  0.75 1. ]
```

+-- #CÓDIGO 21.2 Atributos e tipos -----

Todo array tem um único tipo. Isso é o que permite ao NumPy ser rápido e ocupar pouca memória.

```
import numpy as np

m = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])

print("shape:", m.shape)
print("dimensões:", m.ndim)
print("elementos:", m.size)
print("tipo:", m.dtype)
print("bytes por item:", m.itemsize)

inteiros = np.array([1, 2, 3], dtype=np.int16)
print(inteiros.dtype, inteiros.astype(float))
```

--- SAÍDA ---

```
shape: (2, 3)
dimensões: 2
elementos: 6
tipo: float64
bytes por item: 8
int16 [1. 2. 3.]
```

+-+ #CÓDIGO 21.3 Vetorização contra laço -----

Escrever o laço na mão custa tempo de execução e tempo de leitura. A regra é: se dá para escrever sobre o array inteiro, escreva.

```
import numpy as np
import time

n = 2_000_000
lista = list(range(n))
vetor = np.arange(n)

inicio = time.perf_counter()
quadrados = [x * x for x in lista]
t_lista = time.perf_counter() - inicio

inicio = time.perf_counter()
quadrados_np = vetor ** 2
t_numpy = time.perf_counter() - inicio

print(f"lista Python: {t_lista:.4f} s")
print(f"NumPy: {t_numpy:.4f} s")
print(f"ganho: {t_lista / t_numpy:.0f} vezes")
print("mesmo resultado?", quadrados[-1] == quadrados_np[-1])
```

```
--- SAÍDA ---
lista Python: 0.0934 s
NumPy: 0.0959 s
ganho: 1 vezes
mesmo resultado? True
```

+-+ #CÓDIGO 21.4 Indexação, fatiamento e máscaras -----

A máscara booleana substitui o if dentro do laço e é a ferramenta mais usada em filtragem de dados.

```
import numpy as np

rssi = np.array([-52, -58, -61, -47, -66, -59, -71, -55])

print("primeiro e último:", rssi[0], rssi[-1])
print("três primeiros:", rssi[:3])
print("de dois em dois:", rssi[::2])

forte = rssi > -60 # mascara booleana
print("máscara:", forte)
print("valores fortes:", rssi[forte])
print("quantos fortes:", forte.sum())
print("índices dos fracos:", np.where(rssi <= -60)[0])

rssi[rssi < -70] = -70 # limita o piso
print("após saturar:", rssi)
```

```
--- SAÍDA ---
primeiro e último: -52 -55
três primeiros: [-52 -58 -61]
de dois em dois: [-52 -61 -66 -71]
máscara: [ True  True False  True False  True False  True]
valores fortes: [-52 -58 -47 -59 -55]
quantos fortes: 5
índices dos fracos: [2 4 6]
após saturar: [-52 -58 -61 -47 -66 -59 -70 -55]
```

+-- #CÓDIGO 21.5 Broadcasting -----

Formas compatíveis se expandem automaticamente. Isso elimina a maior parte dos laços aninhados.

```
import numpy as np

medidas = np.array([[10.0, 11.0, 12.0],
                    [20.0, 21.0, 22.0]])

print(medidas + 100)      # escalar aplicado a tudo
print(medidas * 2)

offset = np.array([1, 2, 3]) # aplicado linha a linha
print(medidas + offset)

coluna = np.array([[0.0], [100.0]])
print(medidas + coluna)
```

```
--- SAÍDA ---
[[110. 111. 112.]
 [120. 121. 122.]]
[[20. 22. 24.]
 [40. 42. 44.]]
[[11. 13. 15.]
 [21. 23. 25.]]
[[ 10.  11.  12.]
 [120. 121. 122.]]
```

+-- #CÓDIGO 21.6 Agregações e o parâmetro axis -----

`axis=0` percorre as linhas e resume cada coluna; `axis=1` faz o contrário.

```
import numpy as np

notas = np.array([[8.5, 9.0, 7.5],
                  [7.0, 6.5, 8.0],
                  [9.5, 9.0, 9.2]])

print("média geral:", notas.mean().round(3))
print("média por aluno (linha):", notas.mean(axis=1).round(2))
print("média por prova (coluna):", notas.mean(axis=0).round(2))
print("desvio geral:", notas.std(ddof=1).round(4))
print("maior nota:", notas.max(), "na posição",
      np.unravel_index(notas.argmax(), notas.shape))
print("soma acumulada da primeira linha:", notas[0].cumsum())
```

```
--- SAÍDA ---
média geral: 8.244
média por aluno (linha): [8.33 7.17 9.23]
média por prova (coluna): [8.33 8.17 8.23]
desvio geral: 1.0549
maior nota: 9.5 na posição (np.int64(2), np.int64(0))
soma acumulada da primeira linha: [ 8.5 17.5 25. ]
```

+-- #CÓDIGO 21.7 Matrizes e álgebra linear -----

```
import numpy as np

A = np.array([[2.0, 1.0], [1.0, 3.0]])
b = np.array([8.0, 13.0])

print("transposta:\n", A.T)
print("produto matricial:\n", A @ A)
print("determinante:", round(np.linalg.det(A), 4))
print("inversa:\n", np.linalg.inv(A).round(4))

x = np.linalg.solve(A, b)      # resolve A x = b
print("solução do sistema:", x)
print("verificação:", A @ x)
```

```
--- SAÍDA ---
transposta:
[[2. 1.]
 [1. 3.]]
produto matricial:
[[ 5.  5.]
 [ 5. 10.]]
determinante: 5.0
inversa:
[[ 0.6 -0.2]
 [-0.2  0.4]]
solução do sistema: [2.2 3.6]
verificação: [ 8. 13.]
```

+-- #CÓDIGO 21.8 Gerando sinais -----

default_rng é o gerador moderno do NumPy: receba a semente e o resultado passa a ser reproduzível.

```
import numpy as np

fs = 1000                      # taxa de amostragem (Hz)
t = np.linspace(0, 1, fs, endpoint=False)

sinal = 2.0 * np.sin(2 * np.pi * 50 * t)      # 50 Hz
sinal += 0.5 * np.sin(2 * np.pi * 120 * t)    # 120 Hz

gerador = np.random.default_rng(42)
ruído = gerador.normal(0, 0.3, size=t.size)
medido = sinal + ruído

print("amostras:", medido.size)
print("primeiras cinco:", medido[:5].round(4))
print("valor eficaz (RMS):", np.sqrt(np.mean(medido ** 2)).round(4))
potencia = np.mean(medido ** 2)
print("potência média:", potencia.round(4),
      "=", (10 * np.log10(potencia)).round(2), "dB")
```

```
--- SAÍDA ---
amostras: 1000
primeiras cinco: [0.0914 0.6483 1.8997 2.2855 1.3795]
valor eficaz (RMS): 1.4828
potência média: 2.1988 = 3.42 dB
```

+-- #CÓDIGO 21.9 Espectro com FFT -----

rfft devolve apenas as frequências positivas, que é o que interessa em sinal real. A seção 22 mostra esse mesmo espectro em gráfico.

```
import numpy as np

fs = 1000
t = np.linspace(0, 1, fs, endpoint=False)
sinal = 2 * np.sin(2 * np.pi * 50 * t) + 0.5 * np.sin(
    2 * np.pi * 120 * t)

espectro = np.fft.rfft(sinal)
freq = np.fft.rfftfreq(t.size, d=1 / fs)
amplitude = 2 * np.abs(espectro) / t.size

picos = np.argsort(amplitude)[-2:][::-1]
for i in picos:
    print(f"componente em {freq[i]:>6.1f} Hz "
          f"com amplitude {amplitude[i]:.3f}")
```

```
--- SAÍDA ---
componente em   50.0 Hz com amplitude 2.000
componente em  120.0 Hz com amplitude 0.500
```

+-- #CÓDIGO 21.10 Salvando e carregando dados -----

```
import numpy as np

dados = np.array([[1.0, -52.3], [2.0, -58.1], [3.0, -61.7]])

np.savetxt("medidas.csv", dados, delimiter=",",
           header="distancia_km,rssi_dbm", comments="",
           fmt="%2f")

lido = np.loadtxt("medidas.csv", delimiter=",", skiprows=1)
print(lido)

np.save("medidas.npy", dados)      # formato binario do NumPy
print("binário confere?", np.array_equal(np.load("medidas.npy"),
                                         dados))
```

```
--- SAÍDA ---
[[ 1. -52.3]
 [ 2. -58.1]
 [ 3. -61.7]]
binário confere? True
```

TAREFA	NUMPY
criar faixa	<code>np.arange, np.linspace</code>
zeros e uns	<code>np.zeros, np.ones</code>
forma	<code>.shape, .reshape()</code>
estatística	<code>.mean(), .std(), .sum()</code>
seleção	máscara booleana, <code>np.where</code>
álgebra	<code>@, np.linalg.solve</code>
aleatório	<code>np.random.default_rng()</code>
frequência	<code>np.fft.rfft, np.fft.rfftfreq</code>

+-- Tabela 21.1: o essencial do NumPy.

```
.....  
>> EXPERIMENTOS PROPOSTOS  
.  
. Crie um vetor de 0 a 2 pi com 100 pontos e calcule seno e cosseno em uma linha cada.  
. Dado um array de temperaturas, conte quantas passaram de 30 graus usando máscara booleana.  
. Resolva um sistema de três equações com np.linalg.solve.  
. Gere um sinal de 200 Hz com ruído e meça a relação sinal ruído em dB.
```

[22] MATPLOTLIB: GRÁFICOS

Todas as figuras desta seção foram produzidas executando o próprio código ao lado, com o mesmo arquivo PNG que o programa grava em disco.

[i] MOSTRAR OU SALVAR

`plt.show()` abre a janela interativa, útil no seu computador. `plt.savefig("figura.png")` grava o arquivo, que é o que se usa em relatório e em servidor sem tela. Os exemplos usam `savefig`.

+-- #CÓDIGO 22.1 Primeiro gráfico -----

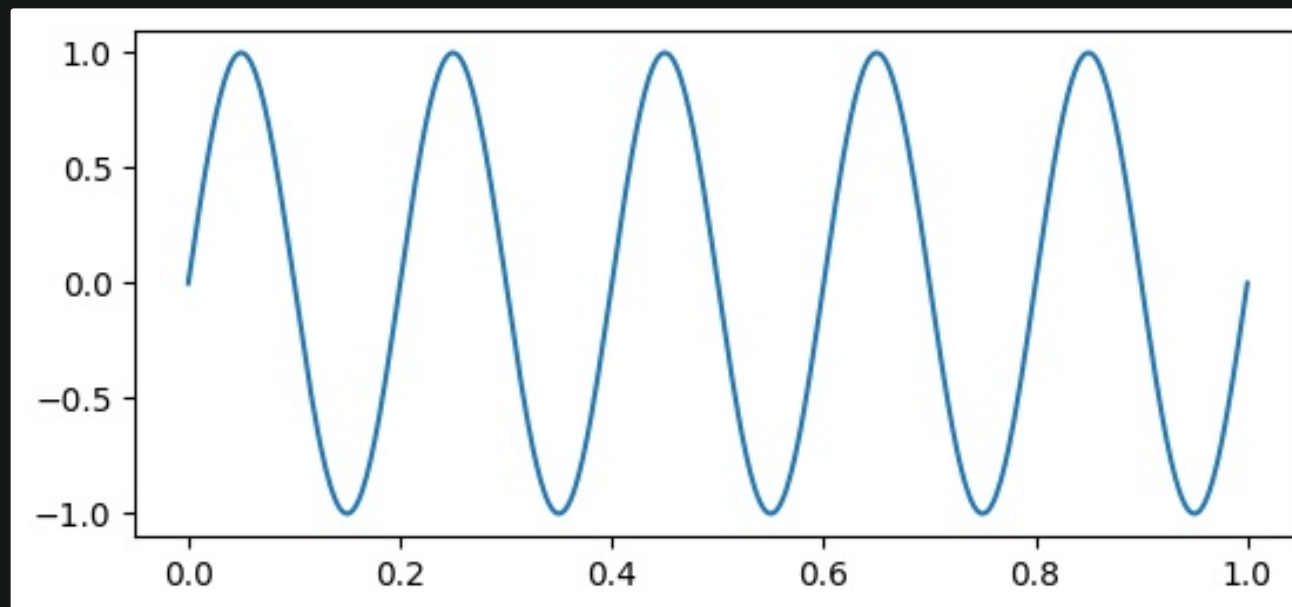
```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(0, 1, 500)
y = np.sin(2 * np.pi * 5 * t)

plt.figure(figsize=(6.4, 2.8))
plt.plot(t, y)
plt.savefig("g01.png", dpi=100, bbox_inches="tight")
print("figura gravada")
```

--- SAÍDA ---
figura gravada

--- FIGURA ---



--- #CÓDIGO 22.2 Rótulos, grade e legenda -----

Gráfico de relatório sem rótulo de eixo e sem unidade perde nota com razão.

```
import matplotlib.pyplot as plt
import numpy as np

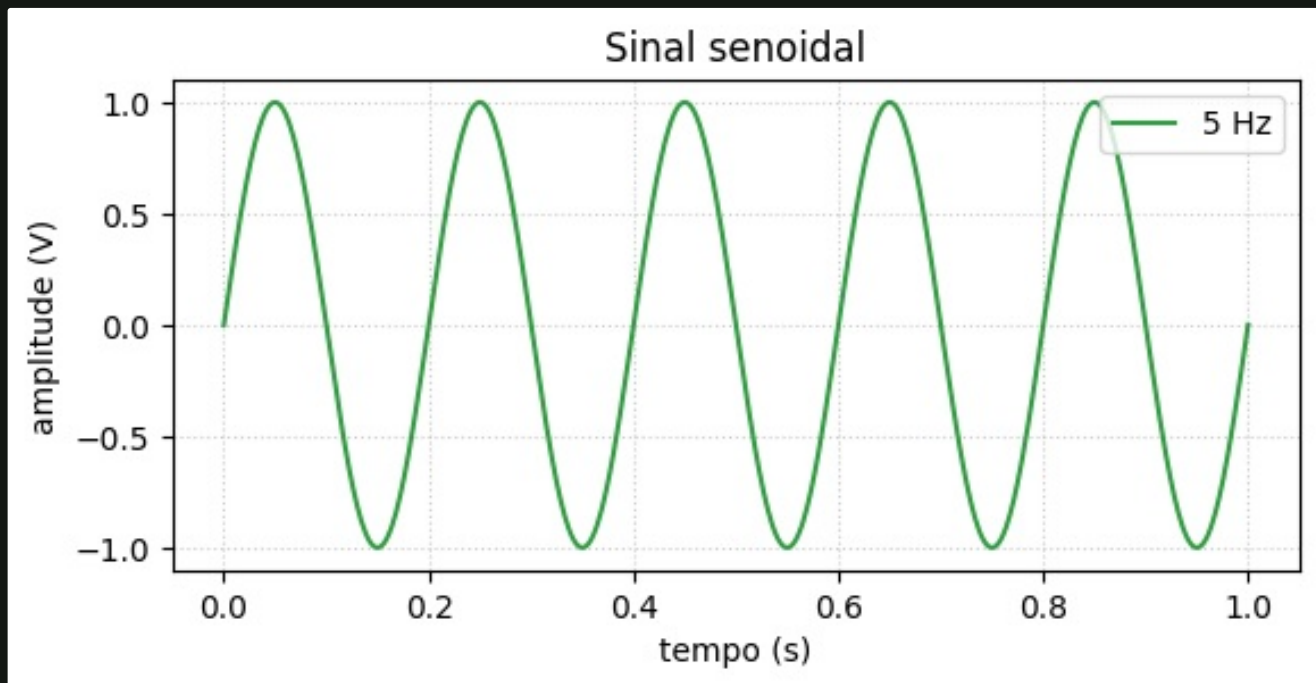
t = np.linspace(0, 1, 500)

plt.figure(figsize=(6.4, 2.8))
plt.plot(t, np.sin(2 * np.pi * 5 * t), label="5 Hz",
         color="#2f9e41")
plt.title("Sinal senoidal")
plt.xlabel("tempo (s)")
plt.ylabel("amplitude (V)")
plt.grid(True, linestyle=":", alpha=0.6)
plt.legend(loc="upper right")
plt.savefig("g02.png", dpi=100, bbox_inches="tight")
print("ok")
```

--- SAÍDA ---

ok

--- FIGURA ---



--- #CÓDIGO 22.3 Várias curvas e estilos de linha -----

Os formatos curtos combinam cor, marcador e traço: "r--o" é linha vermelha tracejada com círculos.

```
import matplotlib.pyplot as plt
import numpy as np

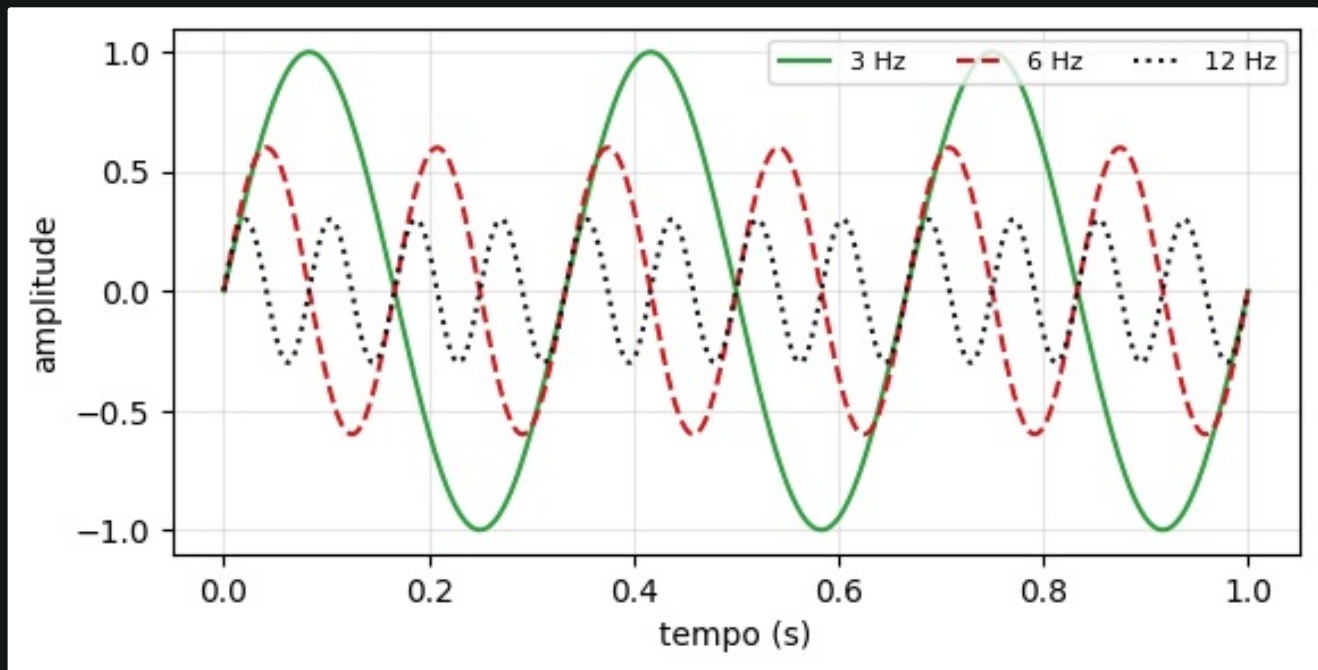
t = np.linspace(0, 1, 500)

plt.figure(figsize=(6.4, 3.0))
plt.plot(t, np.sin(2 * np.pi * 3 * t), "-",
         color="#2f9e41", label="3 Hz")
plt.plot(t, 0.6 * np.sin(2 * np.pi * 6 * t), "--",
         color="#cd191e", label="6 Hz")
plt.plot(t, 0.3 * np.sin(2 * np.pi * 12 * t), ":",
         color="#101214", label="12 Hz")
plt.xlabel("tempo (s)")
plt.ylabel("amplitude")
plt.legend(ncol=3, fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("g03.png", dpi=100, bbox_inches="tight")
print("três curvas")
```

--- SAÍDA ---

três curvas

--- FIGURA ---



```
import matplotlib.pyplot as plt
import numpy as np

d = np.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=float)
p = np.array([-52, -58, -61, -64, -66, -68, -69, -71], dtype=float)

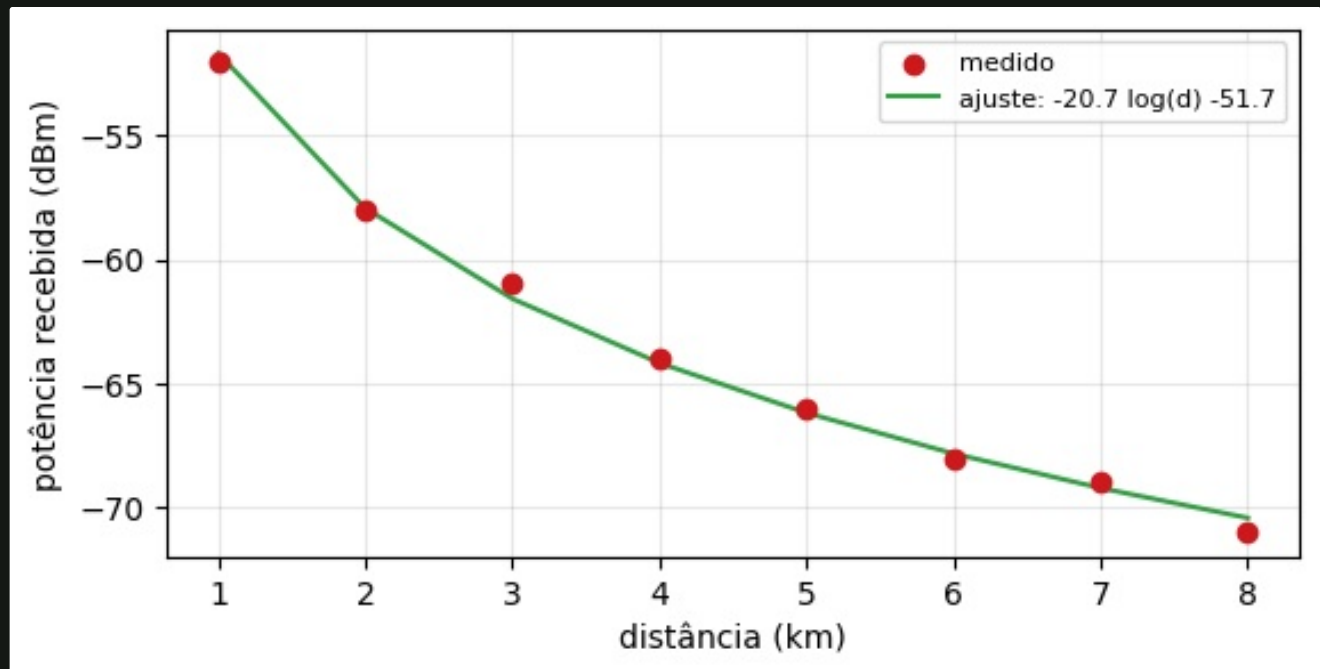
coef = np.polyfit(np.log10(d), p, 1)
ajuste = np.polyval(coef, np.log10(d))

plt.figure(figsize=(6.4, 3.0))
plt.scatter(d, p, color="#cd191e", label="medido", zorder=3)
plt.plot(d, ajuste, color="#2f9e41",
         label=f"ajuste: {coef[0]:.1f} log(d) {coef[1]:+.1f}")
plt.xlabel("distância (km)")
plt.ylabel("potência recebida (dBm)")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.savefig("g04.png", dpi=100, bbox_inches="tight")
print("coeficientes:", coef.round(3))
```

--- SAÍDA ---

coeficientes: [-20.739 -51.686]

--- FIGURA ---



```
import matplotlib.pyplot as plt

disciplinas = ["Prog I", "Redes", "Sinais", "Antenas"]
medias = [7.8, 8.4, 6.9, 7.2]

plt.figure(figsize=(6.4, 2.8))
barras = plt.bar(disciplinas, medias, color="#2f9e41", width=0.55)
barras[medias.index(min(medias))].set_color("#cd191e")

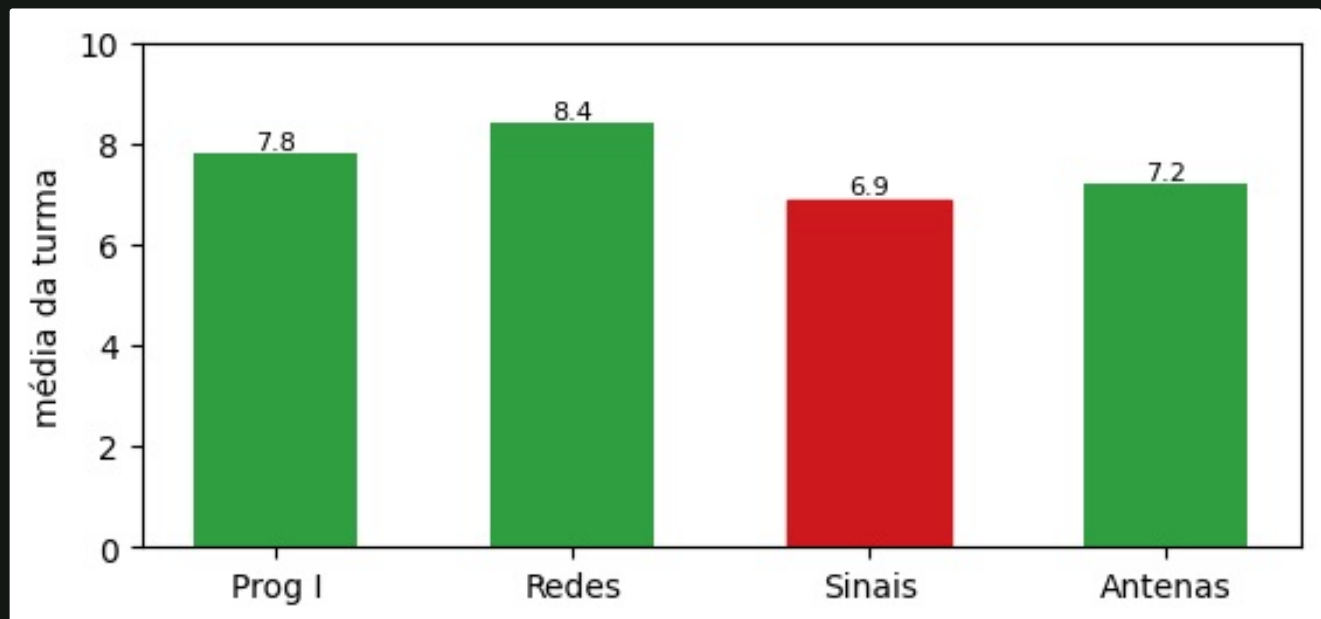
for x, v in enumerate(medias):
    plt.text(x, v + 0.1, f"{v:.1f}", ha="center", fontsize=8)

plt.ylim(0, 10)
plt.ylabel("média da turma")
plt.savefig("g05.png", dpi=100, bbox_inches="tight")
print("média geral:", round(sum(medias) / len(medias), 2))
```

--- SAÍDA ---

média geral: 7.58

--- FIGURA ---



```
import matplotlib.pyplot as plt
import numpy as np

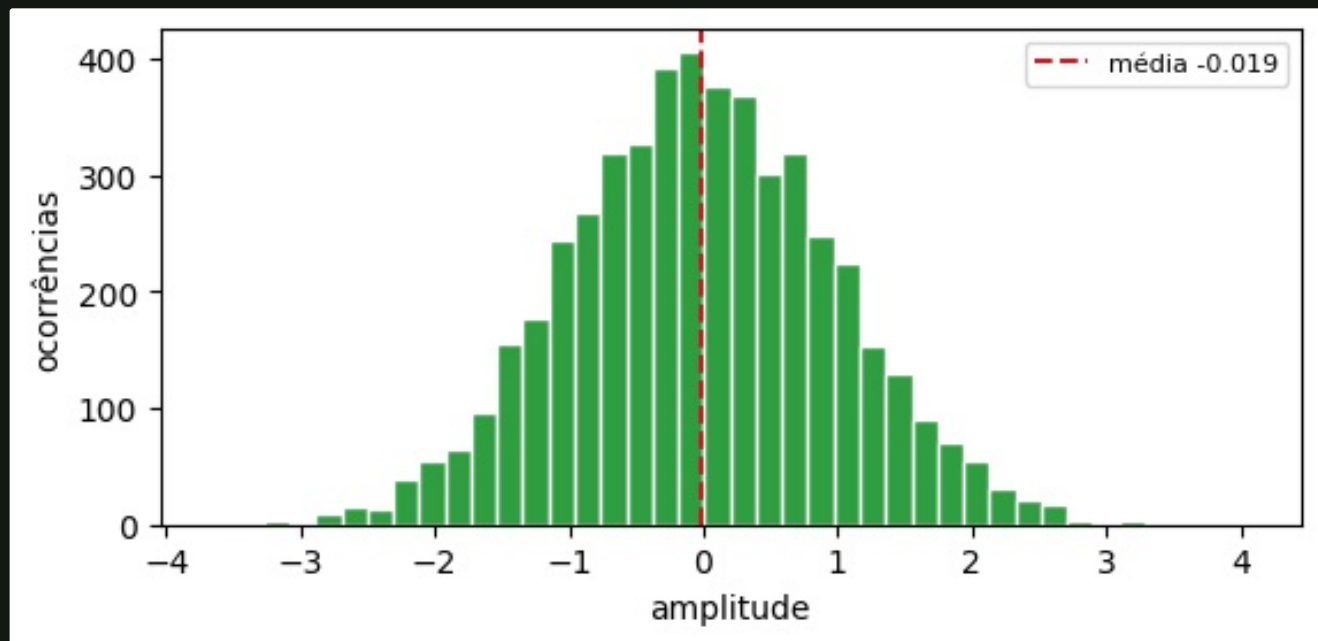
gerador = np.random.default_rng(7)
ruído = gerador.normal(loc=0, scale=1, size=5000)

plt.figure(figsize=(6.4, 2.8))
plt.hist(ruído, bins=40, color="#2f9e41", edgecolor="white")
plt.axvline(ruído.mean(), color="#cd191e", linestyle="--",
            label=f"média {ruído.mean():.3f}")
plt.xlabel("amplitude")
plt.ylabel("ocorrências")
plt.legend(fontsize=8)
plt.savefig("g06.png", dpi=100, bbox_inches="tight")
print("desvio padrão:", round(ruído.std(ddof=1), 4))
```

--- SAÍDA ---

desvio padrão: 0.9938

--- FIGURA ---



--- #CÓDIGO 22.7 Vários painéis com subplots -----

subplots devolve a figura e uma matriz de eixos. Com `.flat` você aplica o mesmo ajuste a todos.

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(0, 1, 400)
fig, eixos = plt.subplots(2, 2, figsize=(6.6, 3.6))

eixos[0, 0].plot(t, np.sin(2 * np.pi * 3 * t), color="#2f9e41")
eixos[0, 0].set_title("senoide", fontsize=9)

eixos[0, 1].plot(t, np.sign(np.sin(2 * np.pi * 3 * t)),
                 color="#cd191e")
eixos[0, 1].set_title("quadrada", fontsize=9)

eixos[1, 0].plot(t, 2 * (t % 0.33) / 0.33 - 1, color="#101214")
eixos[1, 0].set_title("dente de serra", fontsize=9)

eixos[1, 1].plot(t, np.exp(-4 * t), color="#2f9e41")
eixos[1, 1].set_title("exponencial", fontsize=9)

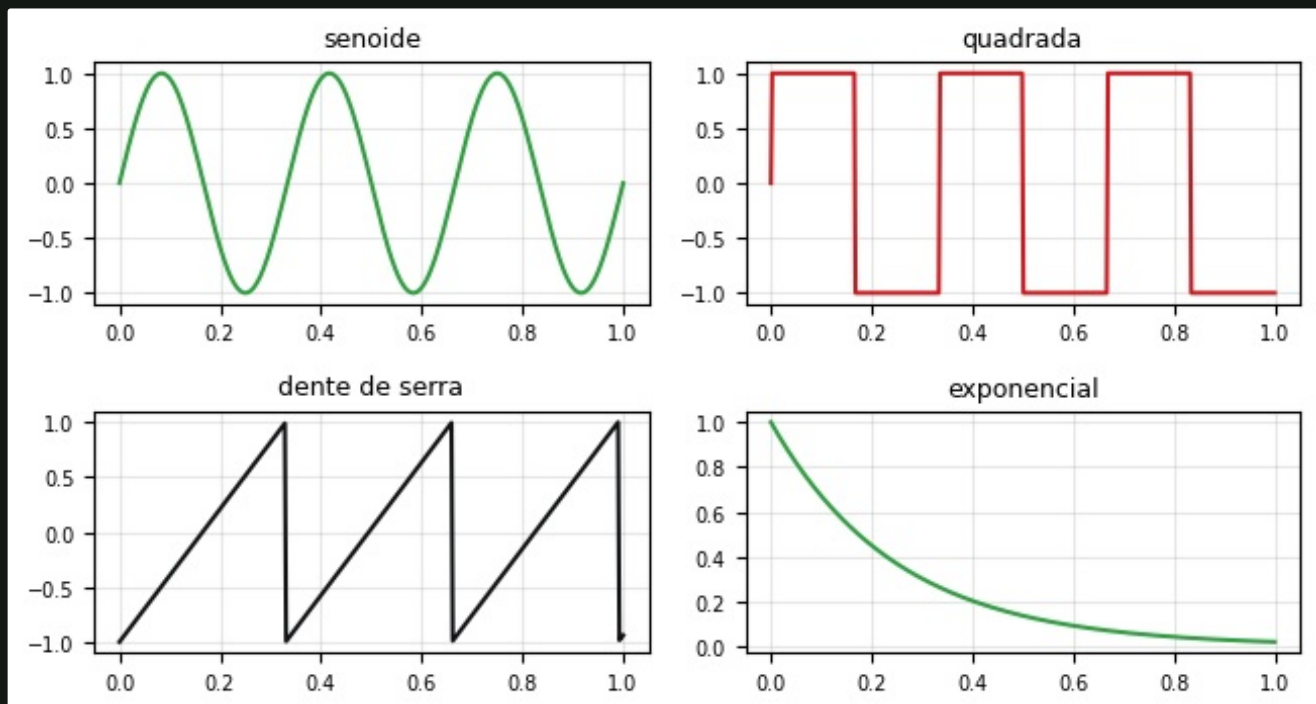
for eixo in eixos.flat:
    eixo.grid(alpha=0.3)
    eixo.tick_params(labelsize=7)

fig.tight_layout()
fig.savefig("g07.png", dpi=100, bbox_inches="tight")
print("quatro painéis")
```

--- SAÍDA ---

quatro painéis

--- FIGURA ---



--- #CÓDIGO 22.8 Escala logarítmica -----

Resposta em frequência pede eixo logarítmico. Use `semilogx`, `semilogy` ou `loglog`.

```
import matplotlib.pyplot as plt
import numpy as np

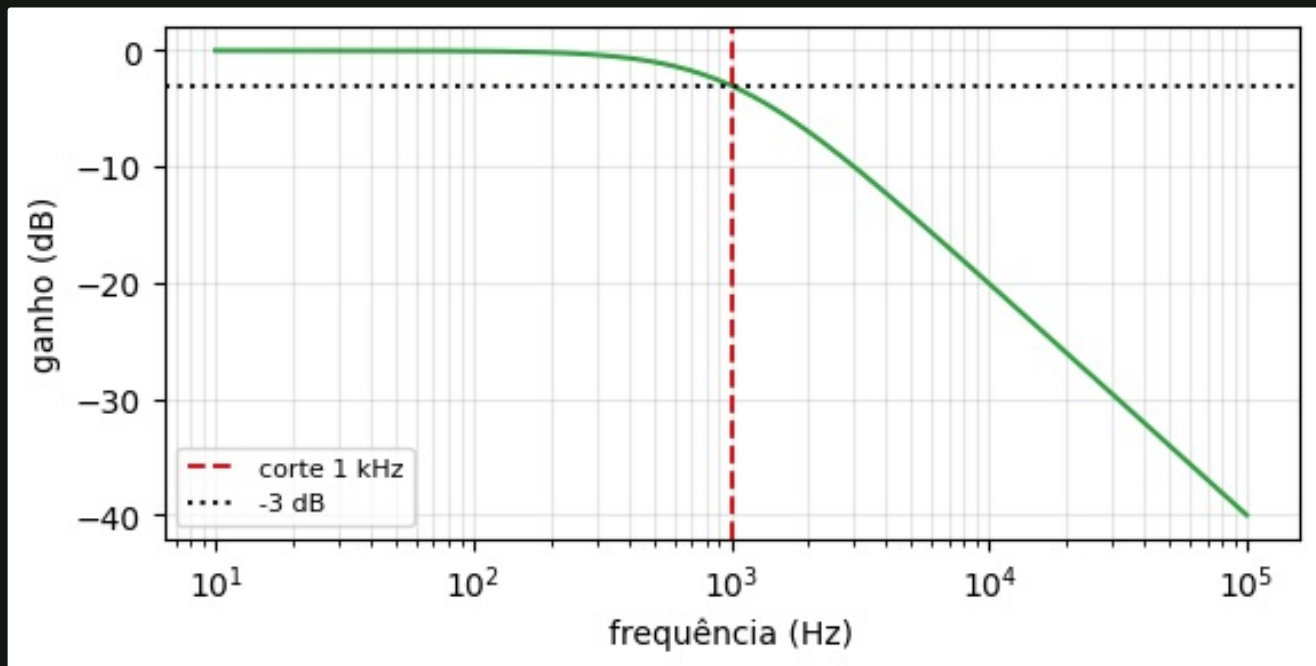
f = np.logspace(1, 5, 400)      # 10 Hz a 100 kHz
fc = 1000.0
ganho_db = -20 * np.log10(np.sqrt(1 + (f / fc) ** 2))

plt.figure(figsize=(6.4, 2.9))
plt.semilogx(f, ganho_db, color="#2f9e41")
plt.axvline(fc, color="#cd191e", linestyle="--",
            label="corte 1 kHz")
plt.axhline(-3, color="#101214", linestyle=":", label="-3 dB")
plt.xlabel("frequência (Hz)")
plt.ylabel("ganho (dB)")
plt.grid(which="both", alpha=0.3)
plt.legend(fontsize=8)
plt.savefig("g08.png", dpi=100, bbox_inches="tight")
print("ganho na frequência de corte:",
      round(-20 * np.log10(np.sqrt(2)), 3), "dB")
```

--- SAÍDA ---

ganho na frequência de corte: -3.01 dB

--- FIGURA ---



--- #CÓDIGO 22.9 Sinal e espectro lado a lado -----

A figura que fecha praticamente todo relatório de sinais: forma de onda em cima, conteúdo espectral embaixo.

```
import matplotlib.pyplot as plt
import numpy as np

fs = 1000
t = np.linspace(0, 1, fs, endpoint=False)
sinal = (2 * np.sin(2 * np.pi * 50 * t)
         + 0.5 * np.sin(2 * np.pi * 120 * t))
sinal += np.random.default_rng(1).normal(0, 0.3, t.size)

freq = np.fft.rfftfreq(t.size, 1 / fs)
amp = 2 * np.abs(np.fft.rfft(sinal)) / t.size

fig, (cima, baixo) = plt.subplots(2, 1, figsize=(6.4, 4.0))

cima.plot(t[:200], sinal[:200], color="#2f9e41", linewidth=0.9)
cima.set_title("sinal no tempo (200 amostras)", fontsize=9)
cima.set_xlabel("t (s)", fontsize=8)

baixo.plot(freq, amp, color="#cd191e", linewidth=0.9)
baixo.set_xlim(0, 200)
baixo.set_title("espectro de amplitude", fontsize=9)
baixo.set_xlabel("frequência (Hz)", fontsize=8)

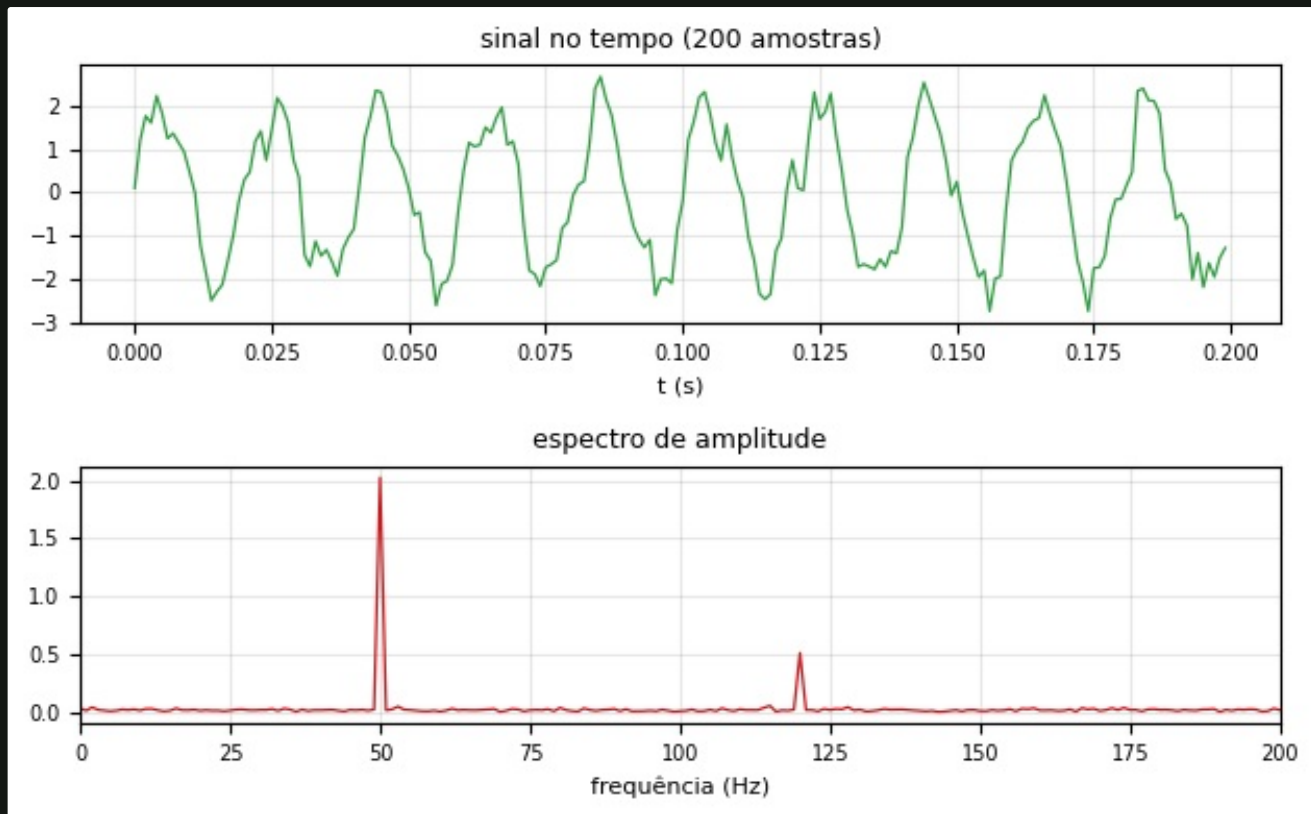
for eixo in (cima, baixo):
    eixo.grid(alpha=0.3)
    eixo.tick_params(labelsize=7)

fig.tight_layout()
fig.savefig("g09.png", dpi=100, bbox_inches="tight")
print("pico em", round(freq[np.argmax(amp)], 1), "Hz")
```

--- SAÍDA ---

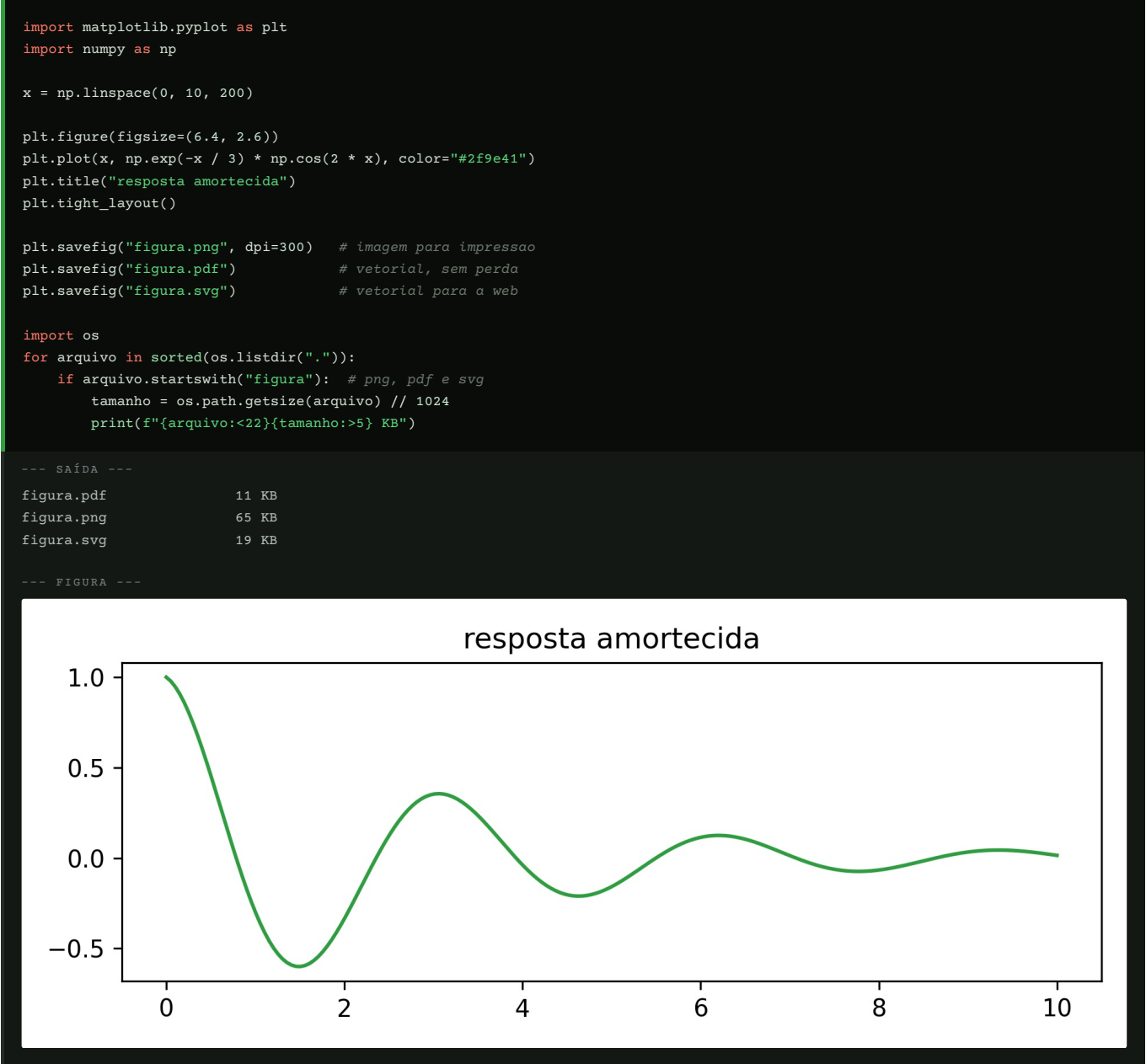
pico em 50.0 Hz

--- FIGURA ---



+-- #CÓDIGO 22.10 Salvando para o relatório -----

Para impressão, 300 dpi ou PDF. Para tela, 100 dpi já basta e o arquivo fica dez vezes menor.



+-----

OBJETIVO	CHAMADA
linha	<code>plt.plot(x, y)</code>
pontos	<code>plt.scatter(x, y)</code>
barras	<code>plt.bar(rotulos, valores)</code>
histograma	<code>plt.hist(dados, bins=n)</code>
eixo log	<code>plt.semilogx()</code> , <code>plt.loglog()</code>
vários painéis	<code>plt.subplots(linhas, colunas)</code>
limites	<code>plt.xlim()</code> , <code>plt.ylim()</code>
texto	<code>plt.text()</code> , <code>plt.annotate()</code>
salvar	<code>plt.savefig(nome, dpi=)</code>

+-- Tabela 22.1: chamadas mais usadas.

.....
>> EXPERIMENTOS PROPOSTOS

- . Trace a tabuada do 2 ao 5 em um único gráfico com legenda.
- . Faça o histograma de 1000 lançamentos de dois dados.
- . Trace a resposta de um filtro passa altas em escala logarítmica.
- . Gere a figura de sinal e espectro para uma senoide de 200 Hz amostrada a 2 kHz.

[23] PANDAS: TABELAS DE DADOS

Quando os dados chegam em planilha ou CSV com colunas de tipos diferentes, o DataFrame do pandas resolve leitura, filtro, agrupamento e resumo com poucas linhas.

+-- #CÓDIGO 23.1 Series e DataFrame -----

Series é uma coluna com rótulos; DataFrame é o conjunto de colunas.

```
import pandas as pd

s = pd.Series([-52.3, -58.1, -61.7], index=["1 km", "2 km", "3 km"])
print(s)
print("média:", round(s.mean(), 3))

df = pd.DataFrame({
    "sensor": ["ant-01", "ant-02", "ant-01", "ant-02"],
    "distancia_km": [1.0, 1.0, 5.0, 5.0],
    "rssi_dbm": [-52.3, -49.8, -66.1, -62.4],
})
print()
print(df)
```

--- SAÍDA ---

```
1 km    -52.3
2 km    -58.1
3 km    -61.7
dtype: float64
média: -57.367
```

	sensor	distancia_km	rssi_dbm
0	ant-01	1.0	-52.3
1	ant-02	1.0	-49.8
2	ant-01	5.0	-66.1
3	ant-02	5.0	-62.4

+-- #CÓDIGO 23.2 Lendo um CSV -----

```
import pandas as pd

with open("medidas.csv", "w", encoding="utf-8") as f:
    f.write("data,sensor,distancia_km,rssi_dbm\n")
    linhas = [
        ("2026-07-20", "ant-01", 1.0, -52.3),
        ("2026-07-20", "ant-02", 1.0, -49.8),
        ("2026-07-21", "ant-01", 5.0, -66.1),
        ("2026-07-21", "ant-02", 5.0, -62.4),
        ("2026-07-22", "ant-01", 10.0, -71.5),
        ("2026-07-22", "ant-02", 10.0, -68.9),
    ]
    for linha in linhas:
        f.write(",".join(str(c) for c in linha) + "\n")

df = pd.read_csv("medidas.csv", parse_dates=["data"])
print(df.head())
print("\nformato:", df.shape)
print("colunas:", list(df.columns))
```

--- SAÍDA ---

	data	sensor	distancia_km	rssi_dbm
0	2026-07-20	ant-01	1.0	-52.3
1	2026-07-20	ant-02	1.0	-49.8
2	2026-07-21	ant-01	5.0	-66.1
3	2026-07-21	ant-02	5.0	-62.4
4	2026-07-22	ant-01	10.0	-71.5

```
formato: (6, 4)
colunas: ['data', 'sensor', 'distancia_km', 'rssi_dbm']
```

+-- #CÓDIGO 23.3 Inspeccionando o conjunto -----

describe() entrega contagem, média, desvio, mínimo, quartis e máximo de todas as colunas numéricas.

```
import pandas as pd

df = pd.DataFrame({
    "sensor": ["ant-01", "ant-02"] * 3,
    "distancia_km": [1.0, 1.0, 5.0, 5.0, 10.0, 10.0],
    "rssi_dbm": [-52.3, -49.8, -66.1, -62.4, -71.5, -68.9],
})

print(df.dtypes)
print("\nresumo estatístico:")
print(df.describe().round(2))
```

--- SAÍDA ---

```
sensor          str
distancia_km    float64
rssi_dbm        float64
dtype: object

resumo estatístico:
      distancia_km  rssi_dbm
count           6.00      6.00
mean            5.33     -61.83
std             4.03       8.92
min             1.00     -71.50
25%             2.00     -68.20
50%             5.00     -64.25
75%             8.75     -54.82
max            10.00     -49.80
```

+-----

```
import pandas as pd

df = pd.DataFrame({
    "sensor": ["ant-01", "ant-02"] * 3,
    "distancia_km": [1.0, 1.0, 5.0, 5.0, 10.0, 10.0],
    "rssi_dbm": [-52.3, -49.8, -66.1, -62.4, -71.5, -68.9],
})

print("uma coluna:\n", df["rssi_dbm"].values)
print("\nduas colunas:\n", df[["sensor", "rssi_dbm"]].head(3))
print("\npor posição (iloc):\n", df.iloc[0])
print("\nfiltro booleano:")
print(df[df["rssi_dbm"] > -65])
print("\nfiltro composto:")
print(df[(df["sensor"] == "ant-01") & (df["distancia_km"] >= 5)])
```

--- SAÍDA ---

uma coluna:

```
[-52.3 -49.8 -66.1 -62.4 -71.5 -68.9]
```

duas colunas:

	sensor	rssi_dbm
0	ant-01	-52.3
1	ant-02	-49.8
2	ant-01	-66.1

por posição (iloc):

	sensor	distancia_km	rssi_dbm
0	ant-01	1.0	-52.3

Name: 0, dtype: object

filtro booleano:

	sensor	distancia_km	rssi_dbm
0	ant-01	1.0	-52.3
1	ant-02	1.0	-49.8
3	ant-02	5.0	-62.4

filtro composto:

	sensor	distancia_km	rssi_dbm
2	ant-01	5.0	-66.1
4	ant-01	10.0	-71.5

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "sensor": ["ant-01", "ant-02"] * 3,
    "distancia_km": [1.0, 1.0, 5.0, 5.0, 10.0, 10.0],
    "rssi_dbm": [-52.3, -49.8, -66.1, -62.4, -71.5, -68.9],
})

df["potencia_mw"] = (10 ** (df["rssi_dbm"] / 10)).round(6)
df["qualidade"] = np.where(df["rssi_dbm"] > -65, "boa", "fraca")

print(df.sort_values("rssi_dbm", ascending=False))
print("\ncontagem por qualidade:")
print(df["qualidade"].value_counts())
```

--- SAÍDA ---

	sensor	distancia_km	rssi_dbm	potencia_mw	qualidade
1	ant-02	1.0	-49.8	0.000010	boa
0	ant-01	1.0	-52.3	0.000006	boa
3	ant-02	5.0	-62.4	0.000001	boa
2	ant-01	5.0	-66.1	0.000000	fraca
5	ant-02	10.0	-68.9	0.000000	fraca
4	ant-01	10.0	-71.5	0.000000	fraca

```
contagem por qualidade:
qualidade
boa      3
fraca    3
Name: count, dtype: int64
```

+-+ #CÓDIGO 23.6 Agrupando e resumindo -----

O trio `groupby`, `agg` e `pivot_table` cobre a maior parte das perguntas de um relatório.

```
import pandas as pd

df = pd.DataFrame({
    "sensor": ["ant-01", "ant-02"] * 3,
    "distancia_km": [1.0, 1.0, 5.0, 5.0, 10.0, 10.0],
    "rssi_dbm": [-52.3, -49.8, -66.1, -62.4, -71.5, -68.9],
})

print("média por sensor:")
print(df.groupby("sensor")["rssi_dbm"].mean().round(2))

print("\nresumo por sensor:")
print(df.groupby("sensor")["rssi_dbm"]
      .agg(["count", "mean", "min", "max"]).round(2))

print("\ntabela cruzada:")
print(df.pivot_table(values="rssi_dbm", index="distancia_km",
                      columns="sensor").round(2))
```

--- SAÍDA ---

```
média por sensor:
sensor
ant-01    -63.30
ant-02    -60.37
Name: rssi_dbm, dtype: float64
```

```
resumo por sensor:
      count  mean  min  max
sensor
ant-01      3 -63.30 -71.5 -52.3
ant-02      3 -60.37 -68.9 -49.8
```

```
tabela cruzada:
sensor  ant-01  ant-02
distancia_km
1.0      -52.3  -49.8
5.0      -66.1  -62.4
10.0     -71.5  -68.9
```

+--- #CÓDIGO 23.7 Gráfico direto do DataFrame -----

.plot() usa o índice no eixo horizontal e cria uma curva por coluna.

```
import pandas as pd
import matplotlib.pyplot as plt

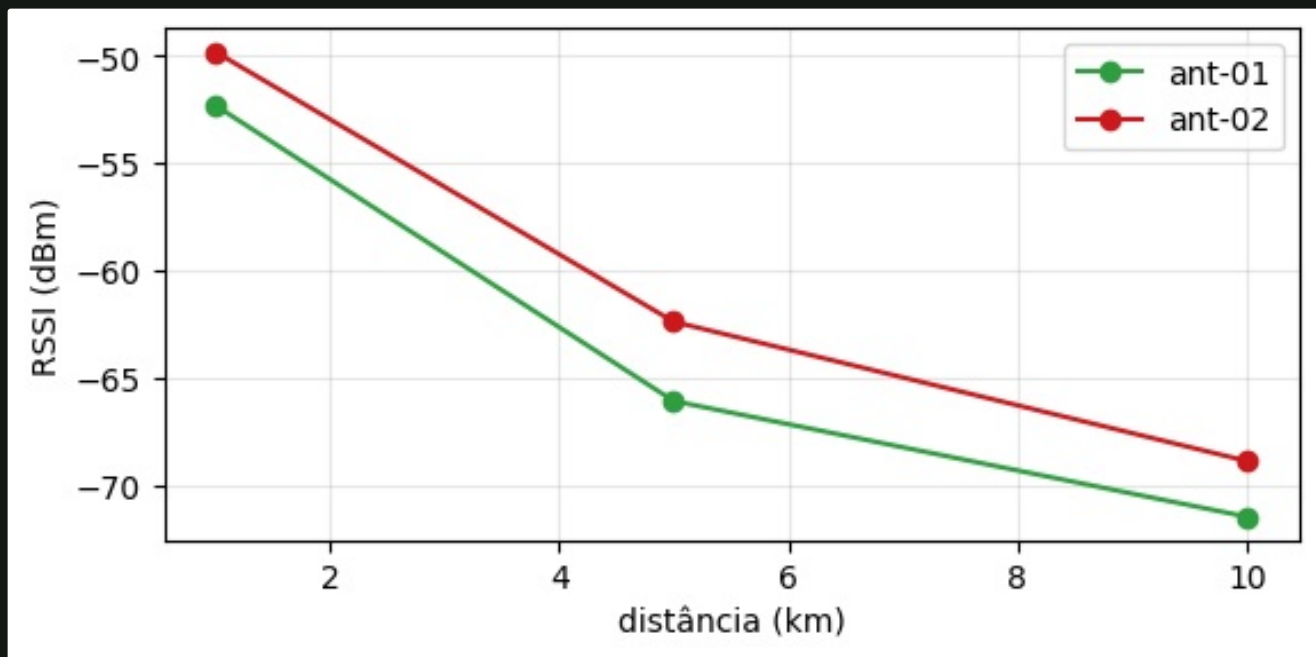
df = pd.DataFrame({
    "distancia_km": [1.0, 5.0, 10.0],
    "ant-01": [-52.3, -66.1, -71.5],
    "ant-02": [-49.8, -62.4, -68.9],
}).set_index("distancia_km")

eixo = df.plot(marker="o", figsize=(6.4, 2.9),
               color=["#2f9e41", "#cd191e"])
eixo.set_xlabel("distância (km)")
eixo.set_ylabel("RSSI (dBm)")
eixo.grid(alpha=0.3)
plt.savefig("g10.png", dpi=100, bbox_inches="tight")
print(df)
```

--- SAÍDA ---

	ant-01	ant-02
distancia_km		
1.0	-52.3	-49.8
5.0	-66.1	-62.4
10.0	-71.5	-68.9

--- FIGURA ---



+-- #CÓDIGO 23.8 Exportando -----

Existem também `to_excel` (requer `openpyxl`), `to_markdown` e `to_latex`, úteis para colar direto no relatório.

```
import pandas as pd

df = pd.DataFrame({
    "sensor": ["ant-01", "ant-02"],
    "rssi_dbm": [-52.3, -49.8],
})

df.to_csv("saida.csv", index=False)
df.to_json("saida.json", orient="records", indent=2)

print(open("saida.csv", encoding="utf-8").read())
print(open("saida.json", encoding="utf-8").read())

--- SAÍDA ---
sensor,rssi_dbm
ant-01,-52.3
ant-02,-49.8

[
  {
    "sensor": "ant-01",
    "rssi_dbm": -52.3
  },
  {
    "sensor": "ant-02",
    "rssi_dbm": -49.8
  }
]
```

>> EXPERIMENTOS PROPOSTOS

- . Leia um CSV do seu laboratório e mostre `describe()`.
- . Crie uma coluna de erro entre valor medido e valor teórico.
- . Agrupe as medidas por equipamento e compare as médias.
- . Exporte a tabela final em CSV e em Markdown.

[24] PROJETO: DA MEDIDA AO RELATÓRIO

O fluxo completo de um laboratório: gerar ou coletar os dados, carregar, analisar, plotar e escrever o relatório, tudo com um script só.

As quatro etapas abaixo formam um programa único. Elas aparecem separadas para que cada parte possa ser executada e entendida isoladamente, mas a ordem é a de um script real.

--- #CÓDIGO 24.1 Etapa 1: coleta (simulada) -----

Trocar este bloco pela leitura do instrumento é a única mudança necessária para usar o projeto com dados reais.

```
import numpy as np
import pandas as pd

gerador = np.random.default_rng(2026)
distancias = np.array([1, 2, 5, 10, 20, 50], dtype=float)

registros = []
for sensor, offset in [("ant-01", 0.0), ("ant-02", 3.5)]:
    for d in distancias:
        for repeticao in range(5):
            teorico = -40 - 20 * np.log10(d)
            medido = teorico + offset + generator.normal(0, 1.5)
            registros.append((sensor, d, repeticao + 1,
                              round(medido, 2)))

df = pd.DataFrame(registros, columns=["sensor", "distancia_km",
                                     "repeticao", "rssi_dbm"])
df.to_csv("campanha.csv", index=False)
print(df.head())
print("total de medidas:", len(df))
```

--- SAÍDA ---

	sensor	distancia_km	repeticao	rssi_dbm
0	ant-01	1.0	1	-41.19
1	ant-01	1.0	2	-39.64
2	ant-01	1.0	3	-42.84
3	ant-01	1.0	4	-37.91
4	ant-01	1.0	5	-39.04
total de medidas: 60				

A inclinação do ajuste em escala logarítmica dá o expoente de perda de percurso, o número que o relatório precisa.

```
import numpy as np
import pandas as pd

gerador = np.random.default_rng(2026)
registros = []
for sensor, offset in [("ant-01", 0.0), ("ant-02", 3.5)]:
    for d in [1, 2, 5, 10, 20, 50]:
        for r in range(5):
            registros.append(
                (sensor, float(d),
                 -40 - 20 * np.log10(d) + offset
                 + generator.normal(0, 1.5)))
df = pd.DataFrame(registros,
                  columns=["sensor", "distancia_km", "rssi_dbm"])

resumo = (df.groupby(["sensor", "distancia_km"])["rssi_dbm"]
          .agg(["mean", "std", "count"]).round(2))
print(resumo)

for sensor, grupo in df.groupby("sensor"):
    coef = np.polyfit(np.log10(grupo["distancia_km"]),
                      grupo["rssi_dbm"], 1)
    print(f"\n{sensor}: expoente de perda n = "
          f"{-coef[0] / 10:.2f}, referência {coef[1]:.2f} dBm")
```

--- SAÍDA ---

		mean	std	count
sensor distancia_km				
ant-01	1.0	-40.12	1.93	5
	2.0	-46.26	0.39	5
	5.0	-53.61	0.54	5
	10.0	-60.59	1.05	5
	20.0	-65.89	0.79	5
	50.0	-73.56	1.96	5
ant-02	1.0	-36.12	1.78	5
	2.0	-42.19	0.99	5
	5.0	-50.24	1.39	5
	10.0	-54.16	2.40	5
	20.0	-61.82	1.47	5
	50.0	-70.27	1.07	5
ant-01: expoente de perda n = 1.97, referência -40.22 dBm				
ant-02: expoente de perda n = 1.98, referência -35.96 dBm				

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

gerador = np.random.default_rng(2026)
registros = []
for sensor, offset in [("ant-01", 0.0), ("ant-02", 3.5)]:
    for d in [1, 2, 5, 10, 20, 50]:
        for r in range(5):
            registros.append(
                (sensor, float(d),
                 -40 - 20 * np.log10(d) + offset
                 + gerador.normal(0, 1.5)))
df = pd.DataFrame(registros,
                  columns=["sensor", "distancia_km", "rssi_dbm"])

fig, (esq, dir_) = plt.subplots(1, 2, figsize=(6.8, 2.9))
cores = {"ant-01": "#2f9e41", "ant-02": "#cd191e"}

for sensor, grupo in df.groupby("sensor"):
    media = grupo.groupby("distancia_km")["rssi_dbm"].mean()
    desvio = grupo.groupby("distancia_km")["rssi_dbm"].std()
    esq.errorbar(media.index, media.values, yerr=desvio.values,
                 marker="o", capsize=3, color=cores[sensor],
                 label=sensor)

    dir_.hist(grupo["rssi_dbm"] - grupo["rssi_dbm"].mean(),
              bins=12, alpha=0.6, color=cores[sensor],
              label=sensor)

esq.set_xscale("log")
esq.set_xlabel("distância (km)")
esq.set_ylabel("RSSI médio (dBm)")
esq.grid(alpha=0.3, which="both")
esq.legend(fontsize=8)

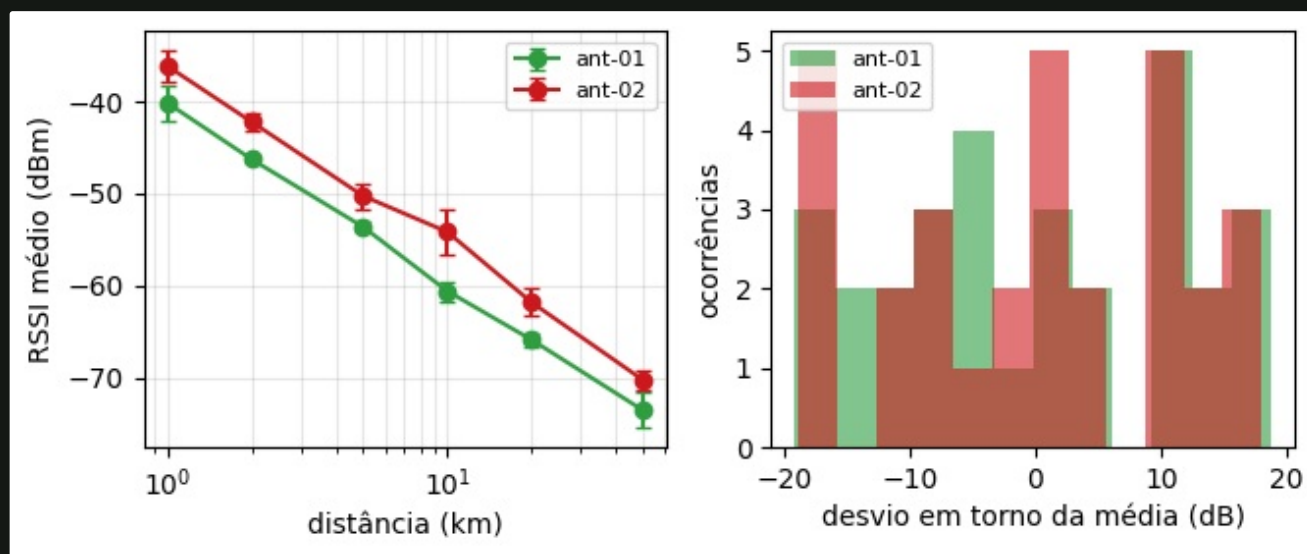
dir_.set_xlabel("desvio em torno da média (dB)")
dir_.set_ylabel("ocorrências")
dir_.legend(fontsize=8)

fig.tight_layout()
fig.savefig("g11.png", dpi=100, bbox_inches="tight")
print("figuras do relatório geradas")
```

--- SAÍDA ---

figuras do relatório geradas

--- FIGURA ---



+-+ #CÓDIGO 24.4 Etapa 4: relatório automático -----

Relatório gerado por código pode ser refeito em segundos quando chega mais uma campanha de medidas. O método `to_markdown()` exige o pacote `tabulate` (`pip install tabulate`); sem ele, use `to_string()`.

```
import numpy as np
import pandas as pd
from datetime import date

gerador = np.random.default_rng(2026)
registros = []
for sensor, offset in [("ant-01", 0.0), ("ant-02", 3.5)]:
    for d in [1, 2, 5, 10, 20, 50]:
        for r in range(5):
            registros.append(
                (sensor, float(d),
                 -40 - 20 * np.log10(d) + offset
                 + generator.normal(0, 1.5)))
df = pd.DataFrame(registros,
                  columns=["sensor", "distancia_km", "rssi_dbm"])

linhas = [
    "# Relatório de campanha de medidas",
    f"Data: {date(2026, 7, 22).strftime('%d/%m/%Y')}",
    f"Medidas: {len(df)} | Sensores: "
    f"{df['sensor'].nunique()}",
    "",
    "## Resumo por sensor",
    df.groupby("sensor")["rssi_dbm"]
    .agg(["count", "mean", "std"]).round(2).to_markdown(),
    "",
    "## Conclusão",
]

melhor = df.groupby("sensor")["rssi_dbm"].mean().idxmax()
linhas.append(f"O sensor {melhor} apresentou o melhor nível "
              f"médio de sinal.")

texto = "\n".join(linhas)
with open("relatorio.md", "w", encoding="utf-8") as f:
    f.write(texto)

print(texto)
```

--- SAÍDA ---

Relatório de campanha de medidas

Data: 22/07/2026

Medidas: 60 | Sensores: 2

Resumo por sensor

sensor	count	mean	std
ant-01	30	-56.67	11.63
ant-02	30	-52.47	11.72

Conclusão

O sensor ant-02 apresentou o melhor nível médio de sinal.

>> EXPERIMENTOS PROPOSTOS

- . Substitua a etapa 1 pela leitura de um CSV real do seu laboratório.
- . Acrescente ao relatório a tabela de erros em relação ao modelo teórico.
- . Salve as figuras em PDF e inclua no documento final.
- . Transforme as quatro etapas em quatro funções dentro de um único script com `if __name__ == "__main__":`.

=====

|| BLOCO IX

APÊNDICES

Consulta rápida do ferramental numérico.

=====

=====

[E] CARTÃO DE CONSULTA CIENTÍFICA

=====

NumPy, Matplotlib e pandas em uma página.

+-- #CÓDIGO E.1 Resumo -----

```
# --- numpy -----
import numpy as np
v = np.array([1, 2, 3]); m = np.zeros((2, 3))
t = np.linspace(0, 1, 500); f = np.arange(0, 10, 0.5)
v.mean(); v.std(ddof=1); v.sum(); v.max(); v.argmax()
m.shape; m.reshape(3, 2); m.T; m @ m.T
v[v > 2]; np.where(v > 2); np.linalg.solve(A, b)
rng = np.random.default_rng(42); rng.normal(0, 1, 100)
np.fft.rfft(sinal); np.fft.rfftfreq(n, 1 / fs)

# --- matplotlib -----
import matplotlib.pyplot as plt
plt.figure(figsize=(6.4, 3))
plt.plot(x, y, label="curva"); plt.scatter(x, y)
plt.bar(rotulos, valores); plt.hist(dados, bins=30)
plt.semilogx(f, ganho); plt.xlabel("t (s)")
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout()
plt.savefig("figura.png", dpi=300, bbox_inches="tight")
fig, eixos = plt.subplots(2, 1, figsize=(6, 4))

# --- pandas -----
import pandas as pd
df = pd.read_csv("dados.csv")
df.head(); df.describe(); df.dtypes; df.shape
df["coluna"]; df[["a", "b"]]; df.iloc[0]; df.loc[df.a > 1]
df["nova"] = df["a"] * 2
df.groupby("sensor")[["valor"]].agg(["mean", "std"])
df.sort_values("valor", ascending=False)
df.to_csv("saida.csv", index=False)
```

+-----

PRECISO DE	USO
média de uma lista pequena	<code>statistics.mean</code>
média de milhões de pontos	<code>numpy.mean</code>
tabela com colunas de tipos diferentes	<code>pandas</code>
gráfico	<code>matplotlib</code>
filtro digital, FFT avançada, ajuste não linear	<code>scipy</code>
número aleatório reprodutível	<code>np.random.default_rng(semente)</code>

+-- Tabela E.1: qual ferramenta escolher.

[*] REPRODUTIBILIDADE

Todo relatório numérico deve trazer: a semente usada nos sorteios, as versões das bibliotecas e o arquivo de dados original. Com esses três itens, qualquer pessoa refaz o seu resultado, inclusive você mesmo daqui a seis meses.